

CEN4025C Module 2 Project

For this project we will walk through creating components for the TaskManager front end. Components are a fundamental building block used by Angular to create dynamic and interactive applications.

Open a Command Prompt window to execute the following:

1. verify npm is installed

```
npm -v
```

2. install angular

```
npm install -g @angular/cli
```

3. verify installation using Angular command line shortcut "ng"

```
ng v
```

4. create the project (accept all defaults – press <Enter> at prompts>

```
ng new TaskManagerFrontEnd
```

5. navigate into the project folder

```
cd TaskMaskManagerFrontEnd
```

6. start VS Code (trust the authors when prompted)

```
code .
```

Explore the environment in VS Code. Some of the main files you will see are:

node_modules this is where the dependencies are loaded when npm install is run. Don't check this folder in to your repo, it is similar to the Maven dependencies which are generated when the project is initially configured and should be recreated each time you clone the repo.

src is where our TypeScript files are, among other things. This is where you will find

index.html the "driver" file for the application (Angular is a single-page framework).

main.ts is the "main method" (in TypeScript) for the application.

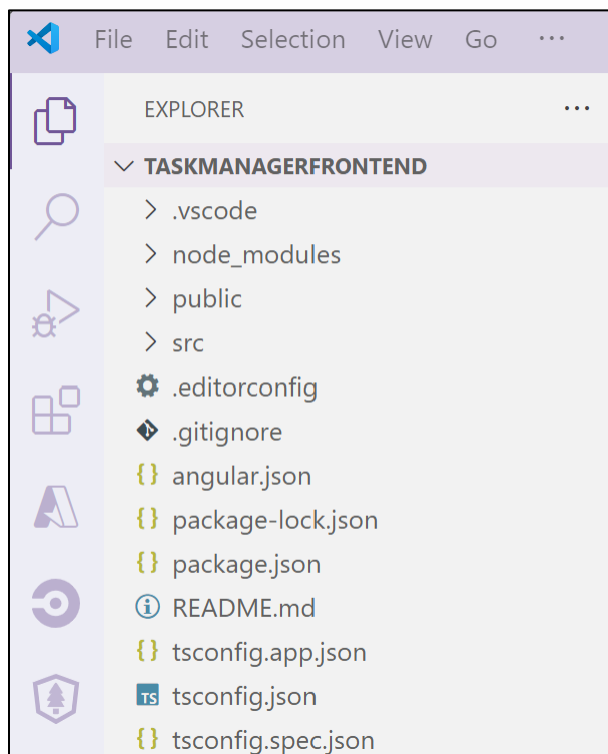
The **app** subfolder is the default component that is created with the project.

package.json is similar to the pom.xml file in Maven, it contains npm dependencies.

package-lock.json is generated when the packages are installed, do not check this into the repo.

angular.json contains the Angular configuration settings

tsconfig.json contains the TypeScript settings.

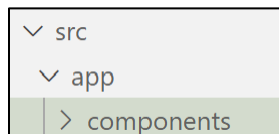


As we saw in Module 1, the program can be run out of the box by running **ng serve** from the console, but the default application is pretty boring.

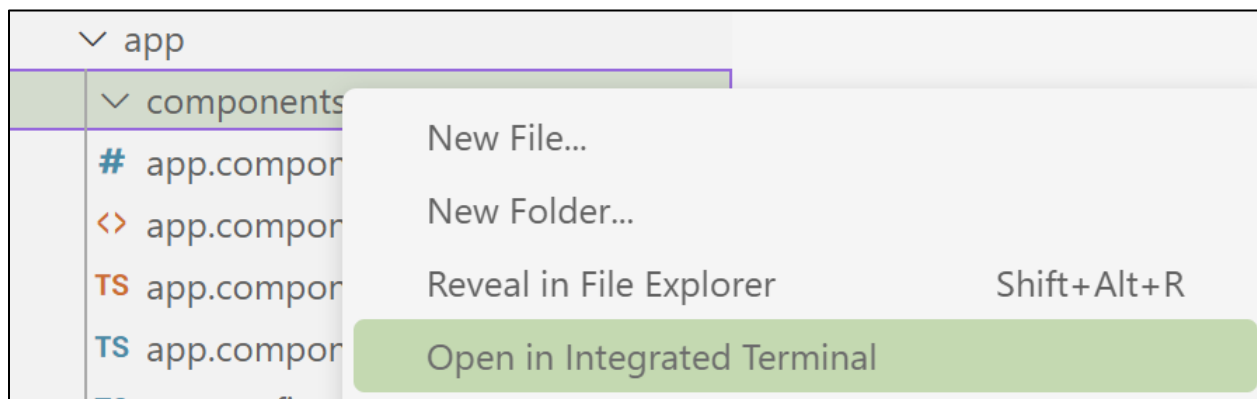
Let's set some groundwork for implementing a front end for the TaskManager application by creating two components: a Tasks component and a Users component (using Tasks and Users as plurals here since these components will track collections of each, vs. singular to manage a single task or user). Our goal is to get a basic front end going that can be extended incrementally.

The easiest way to create a component (and other Angular pieces and parts) is to use the command line from a terminal.

Right-click on the app subfolder and create a new folder name "components":



Right-click on the new components folder and select "Open in Integrated Terminal"



NOTE: Be sure you are using a Command Tool terminal, vs Power Shell, to avoid possible script permissions errors.

Enter the command **ng generate component Tasks**

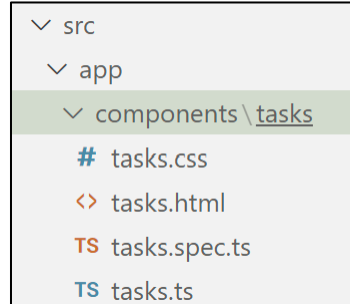
This creates a tasks subfolder under components/ which contains four files:

tasks.css: Styles specific to the component's appearance.

tasks.html: HTML template defining the component's structure.

tasks.spec.ts: Unit test file for the component's functionality

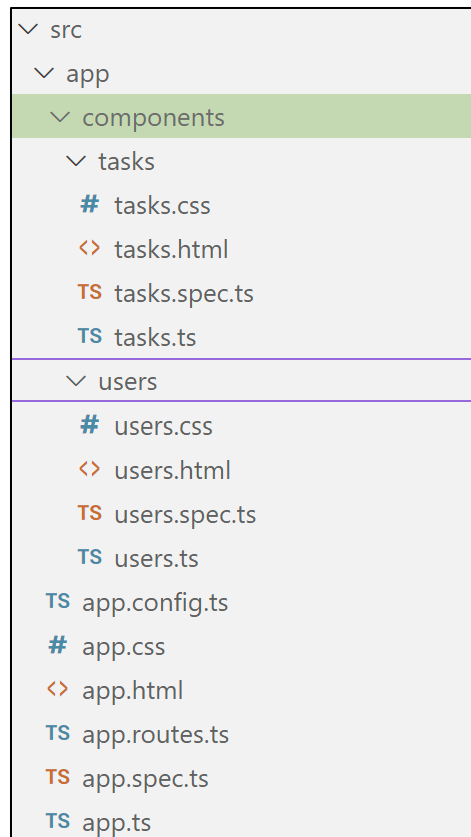
tasks.ts: TypeScript logic and metadata defining the component.



Each component you create will include these files by default.

Now create the Users component. Here's a shortcut that can be used which abbreviates each of the ng arguments if you really hate typing things: **ng g c Users**

Your VS project files should look like this now when you expand the components folder:



Notice the default html that is placed in each of the component's template HTML files:

```
<p>tasks works!</p>
```

```
<p>users works!</p>
```

I'm making minor changes here to show that we can control the content of each component (imagine the more interesting HTML you can provide, which can include style definitions in the associated .css file)

```
<p>Task 1, Task 2, ...</p>
```

```
<p>User 1, User 2, ...</p>
```

Look at the contents of each of the component's TypeScript files; note that they both use a @Component decorator and also define and export a class:

tasks.component.ts:

```
import { Component } from '@angular/core';
```

```
@Component({  
  selector: 'app-tasks',
```

```

    imports: [],
    templateUrl: './tasks.html',
    styleUrls: ['./tasks.css'],
  })
  export class Tasks {
  }

```

users.component.ts:

```

import { Component } from '@angular/core';

@Component({
  selector: 'app-users',
  imports: [],
  templateUrl: './users.html',
  styleUrls: ['./users.css'],
})
export class Users {
}

```

Let's import the classes into the main component. When you add them to the imports array you can let VS Code autocomplete, and it will also automatically add the appropriate import statement. We'll get a warning that the "RouterOutlet" class is not used, but that can be ignored for now.

app.ts:

```

import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Tasks } from './components/tasks/tasks';
import { Users } from './components/users/users';

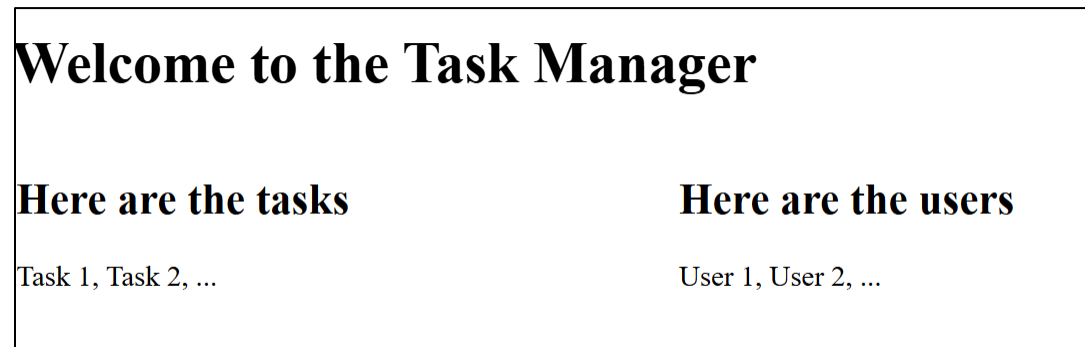
@Component({
  selector: 'app-root',
  imports: [RouterOutlet, Tasks, Users],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  protected readonly title = signal('TaskManagerFrontEnd');
}

```

Finally, let's include the tags for each component in our primary app component's HTML so everything is rendered when we run the application. Replace the default content of app.html with this:

```
<h1>Welcome to the Task Manager</h1>
<table style="width: 100%;">
  <tr>
    <td>
      <h2>Here are the tasks</h2>
      <app-tasks></app-tasks>
    </td>
    <td>
      <h2>Here are the users</h2>
      <app-users></app-users>
    </td>
  </tr>
</table>
```

Run the app and you should see this in your browser:



We will stop here for this version of the project. Submit your project to the GitHub repo and include a screenshot of your final browser view to show that you got it working. Remember that we do not check in the node_modules folder or the package-lock.json file.